

SIAM EV Platform

Truck Device Communication Protocol
STM32H7 · GPS · OBD-II · Dual Camera · 3G/4G

Ruslee Sutthaweeikul

Version 1.0 • July 12, 2026

Contents

1	Overview	3
1.1	Hardware Summary	3
1.2	On-Board Connectivity	3
1.3	Hardware Block Diagram	4
1.4	3G/4G Module Interface	4
1.5	Design Principles	5
2	Transport Layer	5
2.1	MQTT — Telemetry and Events	5
2.2	HTTPS — Photo Upload	5
2.3	Why Not WebSocket Secure (WSS)?	6
2.4	MQTT Topic Structure	6
3	Security	6
3.1	TLS and Device Authentication	6
3.2	Topic-Level Authorisation	7
4	Clock Synchronisation	7
4.1	Sync Procedure on Boot	7
4.2	Timestamp Field Rules	8
5	Telemetry Message — GPS and OBD-II	8
5.1	Publish Interval	8
5.2	Payload Schema	8
5.3	Field Notes	8
5.4	Telemetry Upload Flow	9
6	Station Departure Trigger and Photo Upload	9
6.1	Geo-Fence Logic	9
6.2	Haversine Distance Calculation	9
6.3	Step-by-Step Departure Flow	10
6.4	Error and Retry Handling	11
6.5	Photo Upload Flow	12
7	Offline Buffering	12

8	Connection Management	12
8.1	Reconnect Strategy	12
8.2	MQTT Last Will and Testament	13
8.3	Heartbeat	13
9	Server-to-Device Commands	13
10	Bootstrap and Station Table	14
11	Bandwidth Budget	15
12	Summary of MQTT Topics	15
13	Message Examples	15
13.1	Device Lifecycle	15
13.2	60-Second Telemetry	16
13.3	Station Visit	16
13.4	Server Commands	17

1 Overview

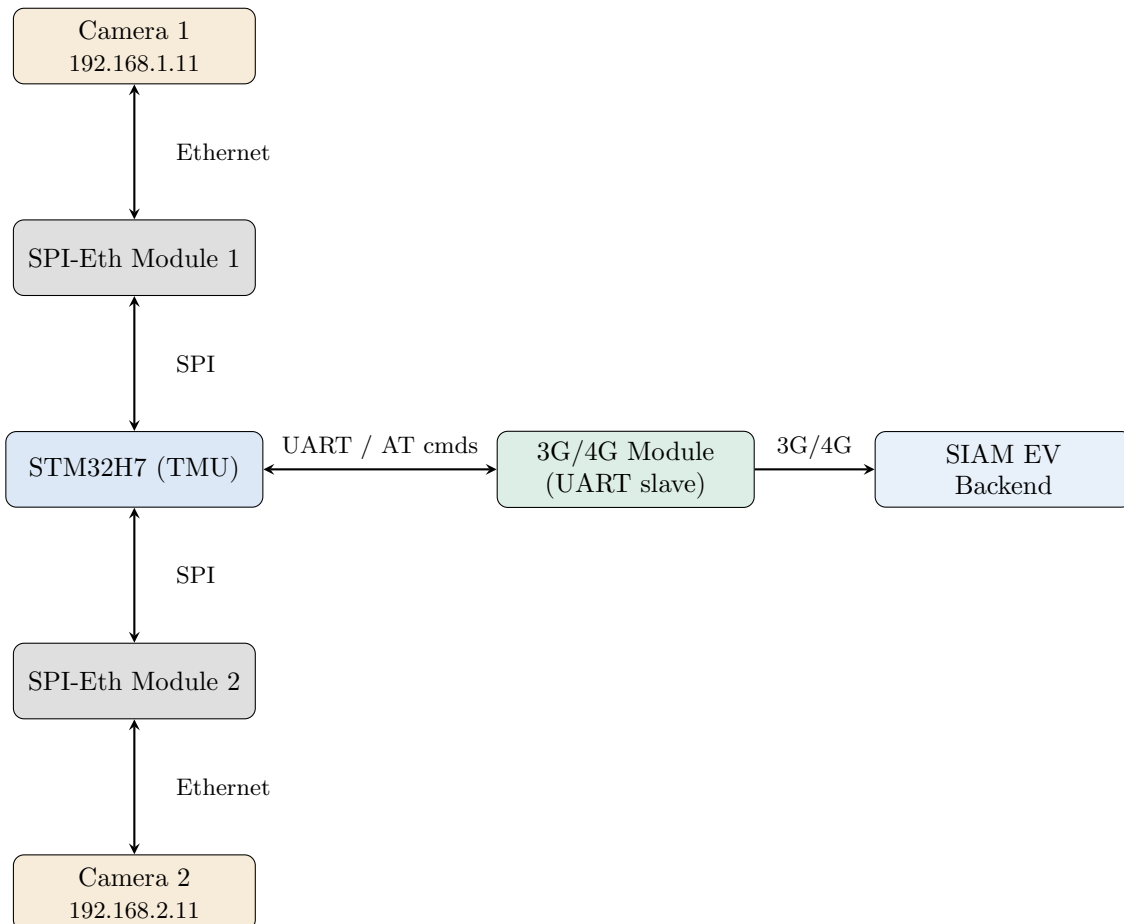
This document specifies the communication protocol between the on-board truck monitoring unit (TMU) and the SIAM EV cargo monitoring backend. It covers message formats, transport configuration, security, and the firmware state machine required to implement correct behaviour on the STM32H7 controller.

1.1 Hardware Summary

Component	Physical interface to MCU	Role
MCU	STM32H7 series	Main controller; runs full protocol and TLS stack
Camera 1	SPI-Eth Module 1	Generic IP camera, cargo front; HTTP snapshot
Camera 2	SPI-Eth Module 2	Generic IP camera, cargo rear; HTTP snapshot
GPS	UART (NMEA 0183)	Position, speed, heading; primary clock source
OBD-II	CAN bus (ISO 15765-4)	Vehicle bus: speed, RPM, fuel, fault codes
3G/4G module	UART (AT commands)	Internet uplink; controlled entirely by STM32H7
SPI-Eth Module 1	SPI	Ethernet interface for Camera 1 (e.g. W5500)
SPI-Eth Module 2	SPI	Ethernet interface for Camera 2 (e.g. W5500)

1.2 On-Board Connectivity

Each IP camera has a dedicated SPI-Ethernet module (e.g. Wiznet W5500) providing an isolated Ethernet segment. There is no shared switch. The 3G/4G module is a UART peripheral — it is not a router and has no IP address on any truck LAN. The STM32H7 is the only device with internet access; it manages the cellular link entirely via AT commands and runs TLS in firmware.



Device	IP address	Notes
STM32H7 (via SPI-Eth 1)	192.168.1.10	eth0; Camera 1 network
Camera 1 (front)	192.168.1.11	Snapshot endpoint: <code>/snapshot.jpg</code>
STM32H7 (via SPI-Eth 2)	192.168.2.10	eth1; Camera 2 network
Camera 2 (rear)	192.168.2.11	Snapshot endpoint: <code>/snapshot.jpg</code>
3G/4G module	n/a	UART peripheral; no IP address

Camera snapshot URL (HTTP GET, point-to-point Ethernet):

```

1 http://192.168.1.11/snapshot.jpg # Camera 1 -- cargo front (SPI-Eth 1)
2 http://192.168.2.11/snapshot.jpg # Camera 2 -- cargo rear (SPI-Eth 2)

```

If a camera requires HTTP Basic Auth, credentials are stored in STM32H7 protected flash and never transmitted off the truck.

1.3 Hardware Block Diagram

Figure 1 shows the physical signal buses connecting all on-board components. Power rails are shown in orange dashed lines; all signal buses are solid bidirectional arrows.

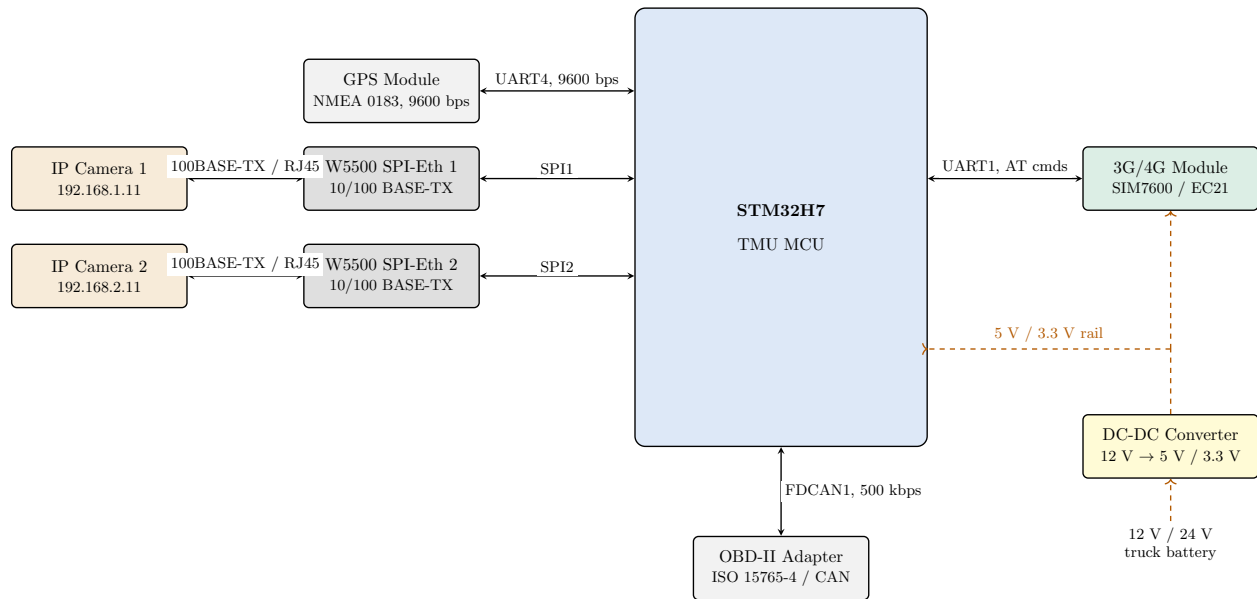


Figure 1: Hardware block diagram. Black: signal buses. Orange dashed: power chain.

1.4 3G/4G Module Interface

The cellular module (e.g. Quectel EC21/EC25, SIM7600, u-blox SARA-R4) is treated as a byte-pipe peripheral. The STM32H7 firmware:

1. **Establishes a PDP context** via AT commands (`AT+CGDCONT`, `ATD*99#` or `AT+CGACT`).
2. **Runs PPP over UART** using the lwIP PPP stack, giving the MCU a routable IP address assigned by the carrier.
3. **Routes all internet traffic** (MQTT, HTTPS, SNTP) through lwIP over the PPP interface. The Ethernet interface serves only the camera LAN and carries no internet traffic.
4. **Runs mbedTLS on the MCU** for all TLS connections. The modem is kept as a dumb data pipe; no SSL AT commands are used. This approach is modem-vendor-independent.

```

1 AT // check modem alive
2 AT+CPIN? // SIM ready check
3 AT+CSQ // signal quality (RSSI -> device.signalDbm)
4 AT+CREG? // network registration
5 AT+CGDCONT=1,"IP","<APN>" // configure PDP context
6 ATD*99***1# // dial; switch UART to PPP data mode
7 // lwIP PPP negotiation begins on same UART from here

```

Listing 1: Modem init sequence (AT commands over UART)

After PPP is up, all subsequent communication uses standard BSD socket calls through lwIP. The AT command UART is not available again until the PPP session is closed (+++ escape then ATH).

1.5 Design Principles

- **Two protocols, two jobs.** MQTT over TLS carries all lightweight telemetry. HTTPS carries photo uploads. Binary blobs are never sent over MQTT.
- **No WebSocket overhead.** MQTT runs directly over TCP + TLS (port 8883). WebSocket Secure (WSS) adds an HTTP upgrade handshake and per-frame header on top of MQTT for no benefit on a device with direct 3G/4G connectivity. See Section 2.3 for detail.
- **Clock sync on every reboot.** The STM32H7 RTC is not battery-backed and drifts across power cycles. The clock is re-synchronised from GPS on every boot, with NTP as fallback. See Section 4.
- **Tolerate poor connectivity.** The TMU buffers telemetry locally when the modem has no signal and flushes on reconnect.
- **Minimal bandwidth.** Telemetry payloads are compact JSON. Photos are JPEG-compressed to ≤ 512 KB before upload.
- **One departure photo per stop.** A geo-fence state machine prevents duplicate uploads when the truck briefly re-enters the 200 m radius.

2 Transport Layer

2.1 MQTT — Telemetry and Events

All telemetry (GPS + OBD) and event notifications use MQTT 3.1.1 over TLS.

Parameter	Value
Protocol	MQTT 3.1.1
Transport	TCP (POC unencrypted) or TLS 1.2 (Production)
Port	1883 (POC) or 8883 (Production)
Broker host	91.210.146.166 (POC) / mqtt.siam-ev.example.com
Client ID	truck-{truckId} (unique per device)
Keep-alive	60 s
QoS (telemetry)	1 (at-least-once)
QoS (photos)	1
Clean session	false (persist subscriptions across reconnects)

2.2 HTTPS — Photo Upload

Photos are uploaded via HTTPS POST to a fixed backend endpoint. The truck authenticates with its provisioned X.509 client certificate (the same identity used for MQTT). No pre-signed URL exchange is required.

Parameter	Value
Protocol	HTTP (POC) / HTTPS (Production mutual TLS)
Method	POST
URL	<code>http://91.210.146.166:3000/v1/trucks/{id}/photos</code> (POC) <code>https://api.siam-ev.example.com/v1/trucks/{id}/photos</code> (Prod)
Content-Type	<code>image/jpeg</code>
X-Station-Id	Station the truck departed from
X-Camera	<code>cargo-front</code> or <code>cargo-rear</code>
X-Timestamp	Unix UTC seconds of the departure
Max payload	512 KB per photo
Timeout	30 s

The server responds with 200 OK and a JSON body `{"photoId": "photo-0041-001"}`. The TMU includes this ID in the subsequent `PhotoReady` MQTT event so the backend can correlate the upload with the trip record.

2.3 Why Not WebSocket Secure (WSS)?

WSS (MQTT over WebSocket over TLS) is sometimes proposed for IoT because port 443 is rarely blocked by enterprise firewalls. For a truck with a dedicated 3G/4G SIM it offers no advantage and several disadvantages:

	MQTT/TLS (port 8883)	MQTT/WSS (port 443)
Protocol layers	TCP + TLS + MQTT	TCP + TLS + HTTP + WebSocket + MQTT
Per-message overhead	MQTT header (2–5 B)	MQTT + WS frame (8–14 B extra)
Handshake cost	TLS handshake	TLS + HTTP upgrade + WS handshake
Firmware code size	mbedTLS + MQTT lib	Above + HTTP parser + WS framer
Firewall concern	None (direct SIM/APN)	None (direct SIM/APN)

Decision: use MQTT/TLS on port 8883. If a future deployment finds that port 8883 is blocked on a specific APN, the correct fix is to configure the MQTT broker to also listen on port 443 — not to add the WebSocket layer.

2.4 MQTT Topic Structure

Topic	Direction	Purpose
<code>trucks/{id}/telemetry</code>	TMU → Server	GPS + OBD every 60 s
<code>trucks/{id}/event</code>	TMU → Server	Departure trigger, door events
<code>trucks/{id}/photo/ready</code>	TMU → Server	Notify server that photo upload is complete
<code>trucks/{id}/cmd</code>	Server → TMU	Server-initiated commands

3 Security

3.1 TLS and Device Authentication

Each TMU is provisioned at the factory with a unique X.509 client certificate signed by the platform's device CA. The certificate's `Common Name` is `truck-{truckId}` and is used as the MQTT Client ID and to authorise publish/subscribe on that truck's topics only.

Item	Value
Device certificate	X.509 v3, RSA 2048 or EC P-256
CA root certificate	Platform Device CA (PEM, stored in protected flash)
Private key storage	STM32H7 TZEN (TrustZone) secure region
TLS cipher suites	TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (preferred)
Certificate expiry	5 years; renewed OTA before expiry

3.2 Topic-Level Authorisation

The MQTT broker enforces that a device authenticated as `truck-X` may only publish to `trucks/X/#` and subscribe to `trucks/X/#`. Cross-device topic access is denied at the broker.

4 Clock Synchronisation

The STM32H7 internal RTC loses its time setting on every power cycle (no battery backup assumed). All telemetry timestamps are UTC Unix seconds; a wrong clock produces corrupted trip logs. The TMU **must** re-synchronise the RTC on every boot before publishing any message.

4.1 Sync Procedure on Boot

Step 1. Wait for GPS time. Parse NMEA sentences on the GPS UART. Accept the UTC time from any sentence that carries a valid UTC field (`GGA`, `RMC`, or `ZDA`). A 3D positional fix is *not* required — the time field is valid as soon as the receiver acquires at least one satellite (typically 30–60 s after power-on). Write the GPS time to the RTC. GPS is accurate to <1 ms and requires no network access. Set `CLK_GPS`.

Step 2. Fallback: NITZ from the cellular network. If no GPS time is available within 90 s of boot (truck inside a building), the modem PPP connection is brought up. Most cellular networks broadcast the current UTC time via the **Network Identity and Time Zone (NITZ)** mechanism during registration. Enable automatic time sync and read the time back with a single AT command:

```

1 AT+CTZU=1           // enable automatic time zone update from network
2 // modem sends unsolicited +CTZE or +CTZV on registration
3 AT+CCLK?            // read: +CCLK: "26/07/12,08:30:00+28"
4                     // format: YY/MM/DD,hh:mm:ssZ (Z = quarters of hour)

```

Listing 2: NITZ clock read via AT commands

NITZ is free — the time is already delivered during the registration exchange and requires no extra UDP round-trip. Write the parsed time to the RTC. Set `CLK_NITZ`. Accuracy is typically $\pm 1\text{--}2$ s (carrier-dependent).

Step 3. Fallback: SNTP over cellular. If the carrier does not provide NITZ (some MVNOs do not), perform one SNTP (RFC 4330) query over the active PPP link:

```

1 NTP_SERVER = pool.ntp.org # or time.google.com

```

Listing 3: SNTP fallback (one-shot UDP query)

SNTP requires a single UDP exchange. The STM32H7 mbedTLS stack is not needed for SNTP; a plain lwIP UDP socket suffices. Set RTC from the NTP timestamp. Set `CLK_NTP`.

Step 4. Mark clock quality. Store a flag in RAM and include it in every telemetry message as `clkSource`: `CLK_GPS`, `CLK_NITZ`, `CLK_NTP`, or `CLK_UNSET` (all sources failed).

Step 5. Continuous GPS discipline. While the GPS has a valid fix, the RTC is re-synchronised every 10 minutes to prevent drift. The `clkSource` field is updated to `CLK_GPS` once GPS sync is achieved regardless of which source was used at boot.

4.2 Timestamp Field Rules

clkSource	ts field	Server behaviour
CLK_GPS	GPS-derived UTC (<1 ms)	Accepted as authoritative
CLK_NITZ	Network time (± 2 s)	Accepted; flagged in trip log
CLK_NTP	NTP-derived UTC (± 1 s)	Accepted; flagged in trip log
CLK_UNSET	0 (epoch)	Server substitutes broker receive time

When `ts = 0` the server replaces it with its own receive timestamp and sets a `tsEstimated: true` flag on the stored record.

5 Telemetry Message — GPS and OBD-II

5.1 Publish Interval

The TMU publishes one telemetry message to `trucks/{id}/telemetry` every **60 seconds** while the ignition is on. If the modem has no signal, the message is written to the local ring buffer (Section 7) and published as soon as connectivity is restored.

5.2 Payload Schema

```

1 {
2   "v": 1, // protocol version
3   "ts": 1751500860, // Unix timestamp (UTC, seconds)
4   "gps": {
5     "lat": 13.8562, // decimal degrees, WGS-84
6     "lng": 100.6211,
7     "altM": 12.5, // altitude metres above sea level
8     "speedKph": 68.2, // from GPS (cross-check with OBD)
9     "headingDeg": 245, // 0-359, degrees true north
10    "hdop": 1.2, // horizontal dilution of precision
11    "satellites": 8,
12    "fixType": "3D" // NoFix | 2D | 3D
13  },
14  "obd": {
15    "speedKph": 68, // PID 0x0D
16    "rpm": 2100, // PID 0x0C
17    "engineLoadPct": 42, // PID 0x04
18    "coolantTempC": 89, // PID 0x05
19    "fuelLevelPct": 67, // PID 0x2F
20    "throttlePct": 35, // PID 0x11
21    "intakeAirTempC": 34, // PID 0x0F (optional)
22    "dtcCount": 0, // PID 0x01 -- number of fault codes
23    "mil": false // malfunction indicator lamp
24  },
25  "device": {
26    "battV": 12.8, // vehicle battery voltage (ADC)
27    "signalDbm": -82, // modem RSSI
28    "seqNo": 1247, // monotonic counter; detects gaps in buffer flush
29    "bufferDepth": 0, // number of unsent records still buffered
30    "clkSource": "GPS" // GPS | NITZ | NTP | UNSET -- see Section 4
31  }
32 }
```

Listing 4: Telemetry message payload

5.3 Field Notes

- `ts` is set from the GPS time-of-fix (authoritative) rather than the MCU RTC, which may drift. If GPS fix is lost, the MCU RTC is used and `fixType` will be "NoFix".

- `obd.speedKph` and `gps.speedKph` should agree within 5 km/h. A persistent large discrepancy indicates a faulty OBD-II connection; the server logs this as a data-quality warning.
- OBD PIDs that are unsupported by the vehicle are omitted from the payload (not sent as `null`) to keep the message small.
- `device.seqNo` increments by 1 each transmission. The server detects any gap and marks intervening records as missing in the trip log.

5.4 Telemetry Upload Flow

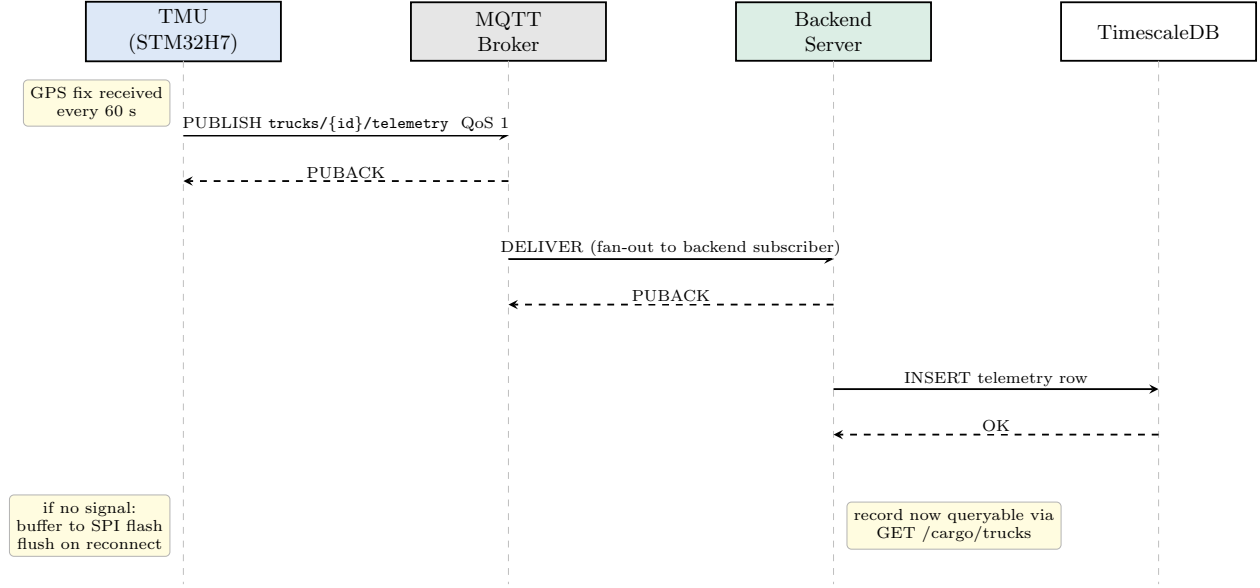


Figure 2: Telemetry upload sequence. QoS 1 ensures the broker stores the message until the backend acknowledges. Offline messages are buffered locally and flushed (5 records/s) once connectivity returns.

6 Station Departure Trigger and Photo Upload

6.1 Geo-Fence Logic

The TMU implements a per-station geo-fence state machine. Station coordinates are downloaded from the server at startup (see Section 10) and stored in flash.

Departure is declared when the truck's GPS position crosses **200 metres** from the station centre, *after* having been inside the radius. This prevents a false trigger when the truck passes near a station without stopping.

6.2 Haversine Distance Calculation

The TMU computes the great-circle distance from the current GPS fix to the station centre using the Haversine formula. This is cheap enough to run on the STM32H7 every GPS fix (once per second while moving).

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\phi_2 - \phi_1}{2} \right) + \cos \phi_1 \cos \phi_2 \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right) \quad (1)$$

where $R = 6,371,000$ m, ϕ = latitude in radians, λ = longitude in radians.

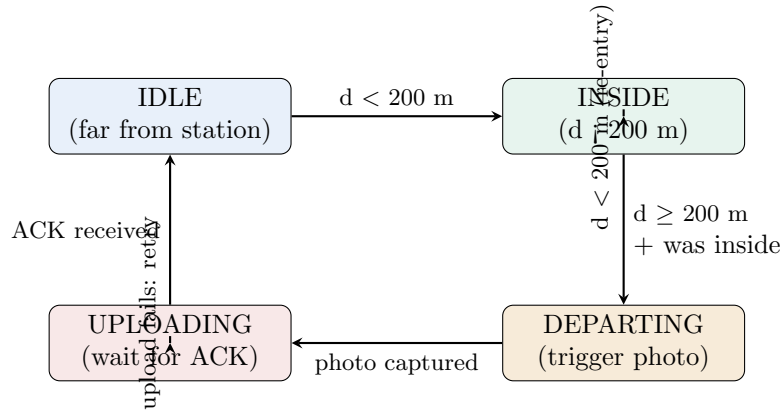


Figure 3: Station departure geo-fence state machine

```

1 #include <math.h>
2 #define EARTH_R_M 6371000.0f
3
4 float haversine_m(float lat1, float lon1, float lat2, float lon2) {
5     float dlat = (lat2 - lat1) * M_PI / 180.0f;
6     float dlon = (lon2 - lon1) * M_PI / 180.0f;
7     float a = sinf(dlat / 2) * sinf(dlat / 2)
8             + cosf(lat1 * M_PI / 180.0f)
9             * cosf(lat2 * M_PI / 180.0f)
10            * sinf(dlon / 2) * sinf(dlon / 2);
11     return 2.0f * EARTH_R_M * asinf(sqrtf(a));
12 }

```

Listing 5: Haversine in C (STM32H7)

6.3 Step-by-Step Departure Flow

Step 1. Transition to DEPARTING. When `haversine_m()` returns ≥ 200 m and the state was `INSIDE`, the TMU transitions to `DEPARTING` and immediately:

- Records `departureTs` (GPS timestamp).
- Records the last GPS fix inside the station (`lastInsideFix`).
- Publishes a departure event to `trucks/{id}/event`.

```

1 {
2     "v": 1,
3     "type": "StationDeparture",
4     "ts": 1751501100,
5     "stationId": "site-bkk-001",
6     "loadPct": 85, // from load sensor at time of departure
7     "gps": {
8         "lat": 13.7302, "lng": 100.7541,
9         "speedKph": 22, "headingDeg": 112
10    }
11 }
12

```

Listing 6: Departure event payload

Step 2. Fetch snapshot from IP camera. Immediately after publishing the departure event the TMU sends an HTTP GET to each camera's snapshot endpoint on the truck LAN:

```

1 GET http://192.168.1.11/snapshot.jpg      # Camera 1 -- cargo front (SPI-Eth 1)
2 GET http://192.168.2.11/snapshot.jpg      # Camera 2 -- cargo rear  (SPI-Eth 2)
3 Timeout: 5 s per camera
4

```

Listing 7: Camera snapshot fetch (on-board LAN)

The cameras return a JPEG directly. No video stream is opened. If the camera does not respond within 5 s, the TMU retries once, then proceeds without that camera's photo and logs `cameraError: true`.

- Expected resolution from camera: 1920×1080 or 1280×960 px (camera-dependent).
- If the raw JPEG exceeds 512 KB, the TMU re-encodes it at reduced JPEG quality (try 70, then 55) using the STM32H7 JPEG hardware encoder until the file fits.
- The TMU writes the GPS timestamp and `truckId` into the EXIF `UserComment` field before upload (preserving the original camera EXIF otherwise).

Step 3. Upload via HTTPS POST. The TMU posts each JPEG to the fixed backend endpoint `POST /v1/trucks/{id}/photos` with context headers (`X-Station-Id`, `X-Camera`, `X-Timestamp`). The server stores the image and responds 200 OK with a JSON body containing the assigned `photoId`. The two cameras upload independently; if one fails, the other proceeds.

Step 4. Publish photo-ready notification. After each successful upload the TMU publishes to `trucks/{id}/photo/ready`.

```

1 {
2   "v":          1,
3   "photoId":    "photo-0041-001",    // assigned by server in upload response
4   "stationId":  "site-bkk-001",
5   "camera":     "cargo-front",       // cargo-front | cargo-rear
6   "ts":         1751501135,
7   "fileSizeKb": 342,
8   "loadPct":    85,
9   "cameraError": false               // true if snapshot fetch failed
10 }
11

```

Listing 8: PhotoReady notification (one per camera)

Both cameras upload independently and each sends its own `PhotoReady` event. The backend links both records under `cargoPhotos`: `[ {camera: "cargo-front", ...}, {camera: "cargo-rear", ...} ]` and emits a `CargoPhotoReady` WebSocket event to the dashboard.

Step 5. Transition to IDLE. On receipt of the server ACK (or after a 30 s timeout) the TMU returns to IDLE and the station departure cycle is complete.

6.4 Error and Retry Handling

Failure	TMU action	Max attempts
No URL received in 10 s	Re-publish departure event	$3 \times$ (then log and skip)
HTTPS PUT timeout / 5xx	Retry with exponential backoff (2 s, 4 s, 8 s)	3
HTTPS PUT 4xx (URL expired)	Re-request URL via departure event	2
Camera capture failure	Log fault, publish event with <code>photoError: true</code>	—

If all retries are exhausted the TMU publishes a `PhotoFailed` event and transitions to IDLE. The trip record on the server will have `cargoPhoto: null` with a `photoError` reason field.

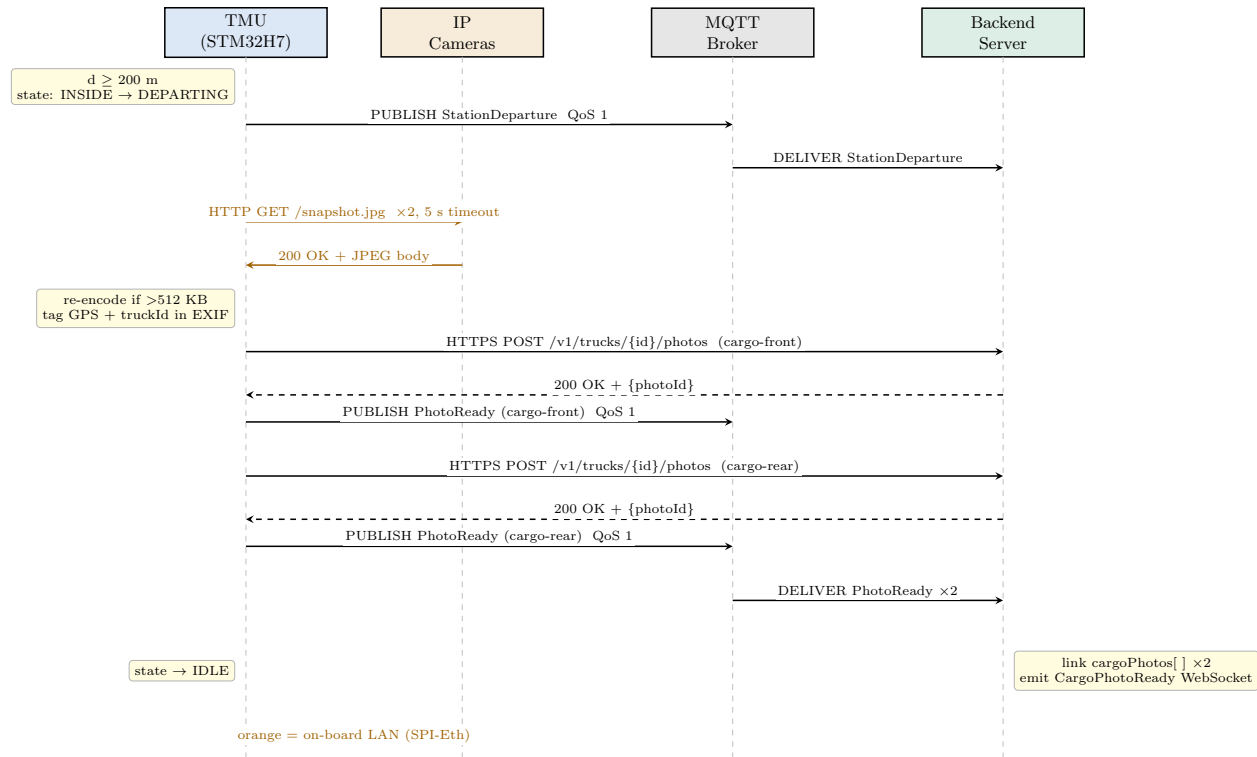


Figure 4: Photo upload sequence. Orange: on-board LAN between TMU and cameras (HTTP over SPI-Ethernet, never leaves the truck). Black solid: MQTT events. Black dashed: HTTPS responses. The truck uploads directly to the backend at a fixed endpoint — no URL is exchanged via the broker.

6.5 Photo Upload Flow

7 Offline Buffering

When the modem loses signal, telemetry must not be silently dropped. The TMU writes unsent records to a FIFO ring buffer in internal or external flash.

Parameter	Value
Buffer medium	External SPI NOR flash (or STM32H7 internal flash sector)
Record size	≤ 256 bytes (one telemetry JSON, compressed if needed)
Buffer capacity	512 records (≈ 8.5 hours at 60 s interval)
Flush order	FIFO — oldest records published first on reconnect
Max flush rate	5 records/s (avoid broker overload on long reconnect)
Buffer overflow	Overwrite oldest records; increment <code>droppedCount</code> counter

The `device.bufferDepth` field in each telemetry message tells the server how many records are still queued on the device. The `device.seqNo` counter allows the server to detect any records that were overwritten and mark them as missing in the trip log.

8 Connection Management

8.1 Reconnect Strategy

```

1 uint32_t backoff_ms = 1000;
2 const uint32_t MAX_BACKOFF_MS = 60000;
3
4 while (!mqtt_connected()) {
5     if (mqtt_connect() == MQTT_OK) {
6         backoff_ms = 1000;    // reset on success
7         flush_buffer();
8         break;
9     }
10    os_delay(backoff_ms);
11    backoff_ms = MIN(backoff_ms * 2, MAX_BACKOFF_MS);
12 }

```

Listing 9: Reconnect backoff (pseudocode)

8.2 MQTT Last Will and Testament

The TMU registers a Last Will message at connect time. The broker publishes it automatically if the client disconnects without sending DISCONNECT:

```

1 {
2     "v": 1,
3     "type": "DeviceOffline",
4     "ts": 0,    // broker fills this with its own timestamp
5     "truckId": "truck-bkk-007"
6 }

```

Listing 10: Last Will payload

Topic: `trucks/{id}/event`, QoS 1, Retain: `false`.

The server marks the truck as `Offline` in the dashboard when this message is received.

8.3 Heartbeat

The MQTT keep-alive (60 s) acts as the heartbeat. No separate application-level ping is required. If the broker does not receive any packet (PUBLISH, PINGREQ, etc.) within $1.5 \times \text{keep-alive} = 90$ s, it closes the connection and triggers the Last Will.

9 Server-to-Device Commands

The server may send commands to the TMU on `trucks/{id}/cmd`. The TMU must acknowledge every command within 10 s.

```

1 {
2     "v": 1,
3     "cmdId": "cmd-20260702-0099",
4     "type": "CapturePhoto",    // see table below
5     "payload": { }
6 }

```

Listing 11: Command envelope

type	payload fields	Action
CapturePhoto	camera (cargo—road)	Capture and upload immediately
UpdateStations	stations (array of Station objects)	Reload geo-fence table
SetInterval	intervalSec (30–300)	Change telemetry interval
Reboot	—	Soft-reboot MCU
FirmwareUpdate	url, sha256	OTA firmware download

Acknowledgement published to `trucks/{id}/event`:

```

1 {
2   "v":      1,
3   "type":   "CmdAck",
4   "cmdId":  "cmd-20260702-0099",
5   "ts":     1751501200,
6   "status": "OK"           // OK | Error
7 }

```

Listing 12: Command acknowledgement

10 Bootstrap and Station Table

On first boot (or after a firmware update) the TMU has no station coordinates. It must request them before the geo-fence logic can operate.

Step 1. TMU connects to MQTT broker.

Step 2. TMU publishes a `Bootstrap` event to `trucks/{id}/event`:

```

1 { "v": 1, "type": "Bootstrap", "ts": 1751500000,
2   "firmware": "1.4.2", "stationTableVersion": 0 }
3

```

Step 3. Server responds with `UpdateStations` command containing the full station table (id, name, lat, lng for every station on the truck's assigned routes).

Step 4. TMU stores the table in flash, sets `stationTableVersion` to the version field in the payload, and enters normal operation.

Step 5. The server re-sends `UpdateStations` whenever the route assignment changes. The TMU checks the version on every boot and requests a refresh if its stored version is stale.

UpdateStations payload — station array

Each element in `payload.stations` is a static reference record containing the fields the TMU needs to run the geo-fence state machine. The geo-fence circle for each station is fully defined by its `lat`/`lng` centre and the protocol-constant 200 m radius (Section 6.1). `name` is stored for diagnostic logs only. The `version` field lets the TMU detect a stale cache on reboot.

```

1 {
2   "v":      1,
3   "cmdId":  "cmd-bootstrap-001",
4   "type":   "UpdateStations",
5   "payload": {
6     "version": 7,
7     "stations": [
8       {
9         "id":    "site-bkk-001",
10        "name":   "Lad Krabang Hub",
11        "lat":    13.7284,
12        "lng":    100.7498
13      },
14      {
15        "id":    "site-bkk-002",
16        "name":   "Don Mueang Depot",
17        "lat":    13.9125,
18        "lng":    100.6068
19      },
20      {
21        "id":    "site-bkk-003",
22        "name":   "Samut Prakan Cold Hub",
23        "lat":    13.5991,
24        "lng":    100.5975

```

```

25     },
26     {
27         "id": "site-bkk-004",
28         "name": "Bang Na Distribution Centre",
29         "lat": 13.6623,
30         "lng": 100.6501
31     }
32 ]
33 }
34 }

```

Listing 13: Topic: trucks/truck-bkk-007/cmd — QoS 1 (server → truck)

On every GPS fix the TMU calls `haversine_m()` against each entry. A station whose distance falls below 200 m triggers the `INSIDE` transition for that geo-fence.

11 Bandwidth Budget

Message type	Size	Frequency	Per hour
Telemetry (GPS + OBD)	≈ 350 B	1/min	≈ 21 KB
Departure event	≈ 200 B	2–6/day	< 1 KB
Photo upload (JPEG)	≤ 512 KB	2–6/day	≤ 3 MB
MQTT keep-alive ping	2 B	1/min	< 1 KB
Daily total (typical)			≈ 25–40 MB

A standard 3G/4G data plan of **1 GB/month per truck** provides more than sufficient headroom. The dominant cost is photo uploads, not telemetry.

12 Summary of MQTT Topics

Topic	QoS	Description
trucks/{id}/telemetry	1	GPS + OBD-II payload, every 60 s
trucks/{id}/event	1	StationDeparture, Bootstrap, CmdAck, DeviceOffline
trucks/{id}/photo/ready	1	Notify server photo upload complete (one per camera)
trucks/{id}/cmd	1	Server-to-device command

13 Message Examples

The examples below use truck ID `truck-bkk-007`. All timestamps are UTC Unix seconds. Arrow direction shows who publishes the message.

13.1 Device Lifecycle

DeviceOnline — truck → server, published on every MQTT connect:

```

1 {
2     "v": 1,
3     "type": "DeviceOnline",
4     "ts": 1751500800,
5     "clkSource": "GPS",
6     "firmware": "1.4.2",
7     "stationTableVersion": 7

```

8 }

Listing 14: Topic: trucks/truck-bkk-007/event — QoS 1

DeviceOffline — auto-published by the *broker* via Last Will when the TCP connection drops without a clean disconnect:

```

1 {
2   "v": 1,
3   "type": "DeviceOffline",
4   "ts": 0
5 }
```

Listing 15: Topic: trucks/truck-bkk-007/event — QoS 1 — retain: true

The `ts: 0` is the Last Will payload set at connect time; the server records the broker receive timestamp instead.

13.2 60-Second Telemetry

Telemetry — truck → server, once per minute while ignition on:

```

1 {
2   "v": 1,
3   "ts": 1751500860,
4   "gps": {
5     "lat": 13.8562,
6     "lng": 100.6211,
7     "altM": 12.5,
8     "speedKph": 68.2,
9     "headingDeg": 245,
10    "hdop": 1.2,
11    "satellites": 8,
12    "fixType": "3D"
13  },
14  "obd": {
15    "speedKph": 68,
16    "rpm": 2100,
17    "engineLoadPct": 42,
18    "coolantTempC": 89,
19    "fuelLevelPct": 67,
20    "throttlePct": 35,
21    "dtcCount": 0,
22    "mil": false
23  },
24  "device": {
25    "battV": 12.8,
26    "signalDbm": -82,
27    "seqNo": 1247,
28    "bufferDepth": 0,
29    "clkSource": "GPS"
30  }
31 }
```

Listing 16: Topic: trucks/truck-bkk-007/telemetry — QoS 1

13.3 Station Visit

StationArrival — truck → server, fired when geo-fence state transitions to **INSIDE** (distance < 200 m):

```

1 {
2   "v": 1,
3   "type": "StationArrival",
4   "ts": 1751501100,
5   "stationId": "site-bkk-001",
6   "distM": 145.3,
7   "tripId": "trip-20260702-0041"
}
```



```
8 }
```

Listing 17: Topic: trucks/truck-bkk-007/event — QoS 1

StationDeparture — truck → server, fired when geo-fence state transitions to **DEPARTING** (distance $\geq$ 200 m after being inside). This is also the trigger for the photo upload flow:

```
1 {
2   "v":      1,
3   "type":   "StationDeparture",
4   "ts":     1751503200,
5   "stationId": "site-bkk-001",
6   "distM":   201.8,
7   "loadPct": 85,
8   "tripId":  "trip-20260702-0041"
9 }
```

Listing 18: Topic: trucks/truck-bkk-007/event — QoS 1

Photo upload — truck → backend, one HTTPS POST per camera to the fixed upload endpoint (no URL exchange required):

```
1 POST http://91.210.146.166:3000/v1/trucks/truck-bkk-007/photos
2 Content-Type: image/jpeg
3 X-Station-Id: site-bkk-001
4 X-Camera:     cargo-front
5 X-Timestamp:  1751503201
6 <binary JPEG body, max 512 KB>
7
8 // Response 200 OK:
9 { "photoId": "photo-0041-001" }
```

Listing 19: HTTPS POST photo upload (repeated for cargo-rear)

PhotoReady — truck → server, after HTTPS POST succeeds for each camera. The **photoId** was returned by the upload response and lets the backend correlate the MQTT event with the stored image:

```
1 {
2   "v":      1,
3   "type":   "PhotoReady",
4   "ts":     1751503245,
5   "photoId": "photo-0041-001",
6   "camera":  "cargo-front",
7   "stationId": "site-bkk-001",
8   "sizeBytes": 398420,
9   "cameraError": false
10 }
```

Listing 20: Topic: trucks/truck-bkk-007/photo/ready — QoS 1

13.4 Server Commands

SetInterval command — server → truck, changing the telemetry publish interval:

```
1 {
2   "v":      1,
3   "cmdId":  "cmd-20260702-0099",
4   "type":   "SetInterval",
5   "payload": { "intervalSec": 30 }
6 }
```

Listing 21: Topic: trucks/truck-bkk-007/cmd — QoS 1

CmdAck — truck → server, within 10 s of receiving the command:

```
1 {
2   "v":      1,
3   "type":   "CmdAck",
4   "cmdId":  "cmd-20260702-0099",
```

```
5  "ts":      1751501200 ,
6  "status": "OK"
7  }
```

Listing 22: Topic: trucks/truck-bkk-007/event — QoS 1

FirmwareUpdate command — server → truck, triggering an OTA download. The truck fetches the binary via HTTPS, verifies the SHA-256 hash, writes to flash, and reboots:

```
1  {
2  "v":      1,
3  "cmdId":  "cmd-20260702-0100",
4  "type":   "FirmwareUpdate",
5  "payload":{
6    "version": "1.5.0",
7    "url":     "http://91.210.146.166:3000/firmware/tmu-1.5.0.bin",
8    "sha256":  "a3f5e2...c9d1"
9  }
10 }
```

Listing 23: Topic: trucks/truck-bkk-007/cmd — QoS 1